INTERPRETABLE TIME SERIES CLASSIFICATION ON A TINY BUDGET: BOTTLENECKED SEPARABLE TCNS WITH TOP-k CONJUNCTIVE POOLING

Anonymous authors
Paper under double-blind review

ABSTRACT

Time series classifiers deployed on sensor nodes are often asked for two things at once: a prediction, and an indication of *which part of the signal produced it*. Multiple instance learning supplies both from a single forward pass, and MILLET (Early et al., 2024) is the current reference formulation. Its cost, however, is set by an InceptionTime backbone: 424 K parameters and 848 MFLOPs per series, one to two orders of magnitude above the flash and compute budgets typical of microcontroller-class hardware. We ask whether the pair *classification + interpretation* survives being made an order of magnitude cheaper. We replace the backbone with a length-preserving multi-scale depthwise-separable dilated convolutional encoder whose pointwise channel mixing — the step that dominates its parameter count — is factorised through a narrow bottleneck, and replace the shared-gate mean of conjunctive pooling with a per-class Top-k head that averages only the κ fraction of most supportive time steps. Neither component is new in isolation; the contribution is the combination and the measurement of what it costs. The resulting model has **41.3 K parameters** and **76.7 MFLOPs** per series, $10.3\times$ and $11.1\times$ below the baseline. On WebTraffic, the only dataset here with per-time-step ground truth, it improves on the equally-trained baseline in both roles at once: accuracy 0.947 against 0.887, NDCG@n 0.772 against 0.677. Across 85 UCR datasets it gives up 1.8 accuracy points against that baseline and 3.4 against MILLET's published recipe, and a wider version of the same encoder recovers the loss — placing the deficit in capacity rather than architecture. An ablation separates the components: the bottleneck removes 28 % of the parameters at no measurable accuracy cost, and Top-k pooling supplies the interpretation gain. We further show that AOPCR, the faithfulness measure used in this literature, moves by $5.2\times$ on a fixed architecture when only the training budget changes, so faithfulness must be compared at matched budgets. We report a parameter and FLOP reduction, not an on-device deployment.

1 INTRODUCTION

A time series classifier is often not the end of the pipeline but the input to a decision. A vibration monitor that reports *bearing fault* is only actionable if an engineer can see which part of the recording caused the verdict; a wearable that reports *arrhythmia* is only auditable if the beats it reacted to can be inspected. Post-hoc attribution (Ribeiro et al., 2016; Lundberg & Lee, 2017; Sundararajan et al., 2017) is one route to that, but it needs a second computation on top of the classifier and explains a surrogate rather than the model. Rudin (2019) argues the alternative: build the explanation in, so that what is explained is what is predicted.

The devices that most need this are the ones least able to afford it. Condition monitoring, wearable health sensing and structural monitoring run on microcontroller-class hardware, where a common configuration is a few hundred kilobytes of SRAM against roughly one megabyte of flash (Lin et al., 2020; Banbury et al., 2021). A model exists there only if its weights fit in flash, its activations fit in SRAM, and its arithmetic fits in the energy budget between two sensor readings. Parameter count and

FLOPs per series are therefore not secondary reporting columns in this setting; they decide whether the method is available at all.

MILLET (Early et al., 2024) is the natural starting point, because it already delivers the pair we want. It casts a series as a *bag* of time steps and applies multiple instance learning (Dietterich et al., 1997; Ilse et al., 2018): an encoder produces one embedding per time step, an attention branch decides where to look, a per-step classifier decides what each position argues for, and the product of the two is both aggregated into the prediction and read out directly as the interpretation. The explanation is not an approximation of the model — it is an intermediate value of the forward pass. MILLET is also strong, matching or beating conventional deep time series classifiers on the UCR archive (Dau et al., 2019).

The cost, however, is inherited from the backbone rather than from the MIL formulation. MILLET's reference configuration uses InceptionTime (Ismail Fawaz et al., 2020): 423,707 parameters and 848 MFLOPs for one series of length 1008, so at 32-bit precision the weights alone occupy about 1.6 MB — above the flash budget above, before activations, runtime or application code. Nothing in the MIL construction requires this. Swapping MILLET's interpretable head for ordinary global average pooling on the same backbone removes only 1,041 of those parameters (Section 3.1): the ability to explain itself costs a quarter of a percent of the model, and essentially all of the cost is the encoder. That observation is what this paper acts on.

We therefore keep the MIL formulation and change the two components that decide its cost and its selectivity. First, the encoder: a stack of multi-scale depthwise-separable dilated convolutions, length-preserving by construction so that one embedding per time step survives to the head, in which the pointwise 1×1 mixing — which dominates the parameter count once the convolutions are separable — is factorised through a narrow bottleneck of width $r = d/4$, halving that term. Second, the pooling head: instead of averaging the evidence over all T time steps behind a single shared attention gate, we score each class separately and average only the $k = \lceil \kappa T \rceil$ largest evidence values for that class. Both mechanisms are established — separable convolutions (Chollet, 2017; Howard et al., 2017), squeeze-and-expand bottlenecks (He et al., 2016; Iandola et al., 2016; Sandler et al., 2018), and k-max selection in MIL (Durand et al., 2016) — and we claim neither as new. What is new is their combination inside an interpretable MIL time series classifier, and the measurement of what that combination retains and what it gives up.

It retains a great deal and gives up something real. The resulting model uses 41,324 parameters and 76.7 MFLOPs per series, $10.3\times$ and $11.1\times$ below MILLET. On WebTraffic, the one dataset here whose generator records which time steps actually carry the class signal, it beats the equally-trained baseline in both roles at once — accuracy 0.947 against 0.887, NDCG@n 0.772 against 0.677 (Figure 1). Across 85 UCR datasets it is worse: 0.810 against 0.827 for that baseline and 0.843 for MILLET's published recipe. We locate that deficit rather than average it away: a wide version of the same encoder, at 269 K parameters, reaches 0.829 and outranks the equally-trained baseline, so what is being paid for is the shrinking and not a defect of the architecture — about two accuracy points for an order of magnitude of cost.

Contributions.

- **A MIL encoder cheap enough to be a candidate for small devices** (Section 3.2): multi-scale depthwise-separable dilated convolutions with a bottlenecked pointwise mix, length-preserving so that per-time-step interpretation survives end to end, at $10.3\times$ fewer parameters and $11.1\times$ fewer FLOPs than MILLET.
- **Per-class Top-k conjunctive pooling** (Section 3.3), which adds no parameters and reduces *exactly* to the baseline head at $\kappa = 1$ (Property 1) — the property that makes the κ sweep of Section 5.4 a controlled experiment rather than a comparison of two models.
- **A component ablation** separating the two changes (Section 5.4): the bottleneck buys the size reduction at no measurable accuracy cost, and Top-k buys the interpretation quality, measured against per-time-step ground truth.
- **A measurement caveat for this literature** (Section 5.5): AOPCR is unnormalised, and on fixed architecture and fixed code it moves by $5.2\times$ when only the training budget changes, so we compare only against baselines we retrained ourselves.

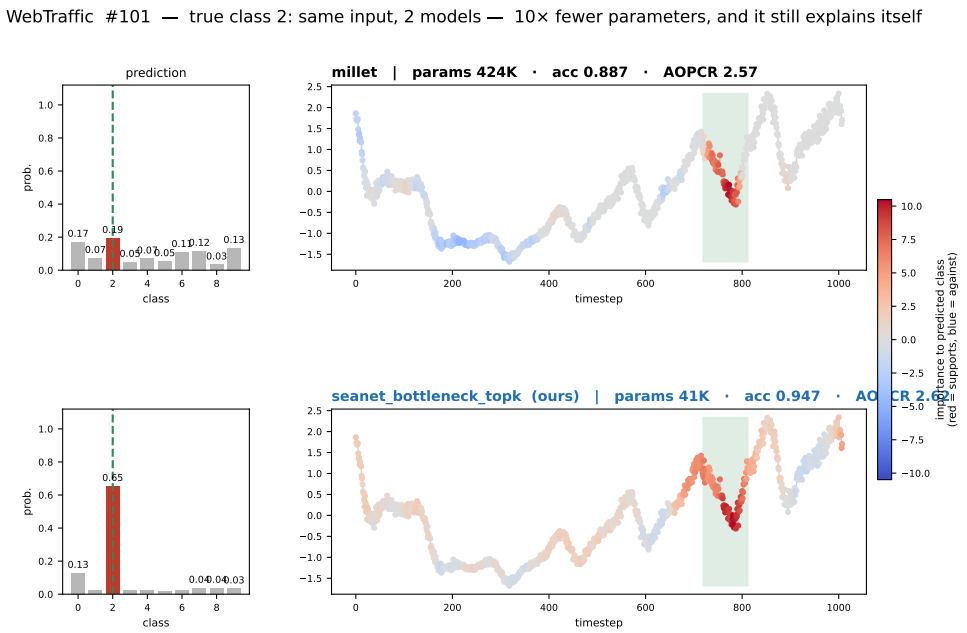

Figure 1: The same WebTraffic series through both models — *top:* the MILLET baseline (423,707 parameters), *bottom:* ours (41,324). Left, the predicted class distribution with the true class marked; right, the series coloured by each model's own interpretation (red supports the predicted class, blue argues against), with the shaded band showing where the generator put the class signal. The smaller model is both more confident and more selective. Panel-title accuracy and AOPCR are dataset means over three seeds; the single series shown is illustrative, not evidence.

We do not deploy on a microcontroller and report no on-device latency, energy or memory: our cost evidence is parameter counts, analytically counted FLOPs and GPU-side timing, which supports *eligibility for a budget* rather than *measured operation within it*. Nor do we claim state of the art on UCR; Section 5.3 quantifies the trade rather than softening it.

2 Related Work

Deep time series classification. Convolutional baselines have dominated the field since FCN and ResNet were shown to be strong without feature engineering (Wang et al., 2017), and InceptionTime (Ismail Fawaz et al., 2020) remains the reference deep architecture on the UCR archive (Dau et al., 2019); the survey of Ismail Fawaz et al. (2019) covers the design space between them. Accuracy in this literature is almost always reported without a cost column, and the models are correspondingly large: those three architectures use between 267 K and 506 K parameters at the input length we study.

Interpretability and multiple instance learning. Attribution methods (Ribeiro et al., 2016; Lundberg & Lee, 2017; Sundararajan et al., 2017) explain a trained model after the fact; Rudin (2019) argues for building the explanation in instead, and for time series VQShape (Wen et al., 2024) and InterpGN (Wen et al., 2025) do so through shapelet-like tokens and gated expert models. MIL (Dietterich et al., 1997) offers a different route, in which the interpretation is an intermediate activation rather than a separate artefact. Attention-based MIL (Ilse et al., 2018) and its dual-stream extension (Li et al., 2021) established the pooling mechanisms in histopathology, and MILLET (Early et al., 2024) transferred the formulation to time series with *conjunctive pooling*, the baseline we start from. Selecting only the strongest instances instead of averaging all of them is likewise established: WELDON (Durand et al., 2016) introduced k-max region selection for weakly supervised recognition, and the idea recurs throughout weakly supervised segmentation.

Efficient architectures and deployment budgets. Depthwise-separable convolution (Chollet, 2017; Howard et al., 2017) and squeeze-and-expand bottlenecks (He et al., 2016; Iandola et al., 2016; Sandler et al., 2018) are the two standard levers for reducing convolutional parameter count, and

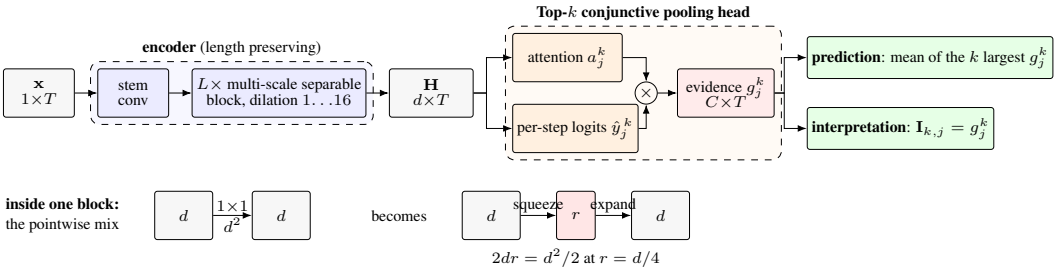

Figure 2: *Top:* the model. The encoder never changes the length, so every downstream quantity keeps a time axis. The head's two branches multiply into the evidence g_j^k, the single quantity from which *both* outputs are read: the prediction is a reduction of it over time, the interpretation *is* it. *Bottom:* the change inside the block — the pointwise 1×1 channel mix is factorised through a narrow middle layer, halving the term that dominates the parameter count.

both transfer directly to the 1-D case. In the time series setting, ATCN (Baharani & Tabkhi, 2022) combines residual connections with separable convolution for embedded inference, and Risso et al. (2023) search the dilation and channel structure of TCNs under edge constraints; neither targets interpretability — they produce a label, and where it came from is left open. On the deployment side, MCUNet (Lin et al., 2020) and MicroNets (Banbury et al., 2021) characterise what fits on commodity microcontrollers (flash for weights, SRAM for activations, operation count as a latency proxy) and both find that architectures designed without those constraints do not simply shrink into them. Post-training integer quantisation (Jacob et al., 2018) is the standard final step and cuts weight memory by roughly four. We use these budgets as the target the paper aims at, not as a result: no measurement here was taken on a microcontroller.

Where this leaves our contribution. Every mechanism we use already exists. What is absent above is the intersection: efficient time series architectures do not produce interpretations, interpretable ones are not built to a cost budget, and the MIL pooling literature evaluates selection by bag-level accuracy rather than against per-instance ground truth. This paper occupies that intersection, contributing specifically the per-class, fraction-parameterised form of k-max selection inside conjunctive pooling with its exact reduction to the baseline head (Property 1); its pairing with a bottlenecked length-preserving encoder; an ablation attributing size and interpretation quality to the two components separately; and the finding that the field's faithfulness measure is not comparable across training budgets.

3 METHOD

3.1 THE BASELINE WE START FROM

A series $\mathbf{X} \in \mathbb{R}^{1\times T}$ is treated as a bag of T instances, one per time step. An encoder f produces $\mathbf{H} = f(\mathbf{X}) \in \mathbb{R}^{d\times T}$, one embedding $\mathbf{z}_j$ per step. MILLET's *conjunctive pooling* then runs two branches over those embeddings and multiplies them:

$$a_j = \frac{\exp\big(w^\top \tanh(V\mathbf{z}_j)\big)}{\sum_i \exp\big(w^\top \tanh(V\mathbf{z}_i)\big)}, \qquad \mathbf{y}_j = W_c\mathbf{z}_j \in \mathbb{R}^C, \qquad g_j^k = a_j\,\mathbf{y}_j^k, \tag{1}$$

where a_j says how much step j matters and $\mathbf{y}_j^k$ says what step j argues for regarding class k. The product g_j^k is the *evidence* of step j for class k, and both outputs are read from it:

$$\hat{y}^k = \frac{1}{T}\sum_{j=1}^{T} g_j^k \qquad \text{(prediction)}, \qquad\qquad \mathbf{I}_{k,j} = g_j^k \qquad \text{(interpretation)}. \tag{2}$$

This is why the explanation is faithful by construction: it is not a model of the model, it is the summand of the prediction. Three properties of Equations (1) and (2) matter later. **(L1)** the gate a_j is shared across classes, so every class is forced to look in the same place; **(L2)** the normalisation in a_j is over time, so attention mass is conserved rather than absolute; and **(L3)** the reduction is a fixed mean over all T steps, so a signal confined to a short window is divided by the length of the whole series.

Where the cost is. The MIL machinery in Equation (1) is almost free. Replacing MILLET's interpretable head with ordinary global average pooling on the same InceptionTime backbone changes the model from 423,707 to 422,666 parameters: explaining itself costs $0.25\,\%$ of the model, and everything else is the encoder. A cheaper interpretable classifier is therefore a cheaper *encoder* problem, not a pooling problem — which is what Section 3.2 addresses, while Section 3.3 addresses quality at a fixed cost.

3.2 A LENGTH-PRESERVING SEPARABLE ENCODER

We replace InceptionTime with a stack of *multi-scale separable residual blocks*. Each unit reads its input with three depthwise convolutions of kernel size $\{5, 11, 23\}$ at a shared dilation, sums them, and mixes channels with a pointwise 1×1 convolution, followed by BatchNorm (Ioffe & Szegedy, 2015), ReLU and dropout; two units and a residual connection form a block, and dilation grows geometrically across blocks up to a cap of 16. Two design constraints are forced by the interpretation rather than by accuracy: convolutions use "same" padding with no stride and no pooling, so T is preserved at every stage — required, because Equation (2) needs one value per input step and the faithfulness metrics perturb individual steps — and the dilation cap keeps every embedding tied to a bounded window, so importance stays local and readable. The third choice is the cost one: the depthwise/pointwise split (Chollet, 2017; Howard et al., 2017) replaces the d^2k weights of a dense convolution with

$$\underbrace{dk}_{\text{depthwise}} + \underbrace{d^2}_{\text{pointwise}} \ll d^2k, \tag{3}$$

which is what makes the small setting reachable at all: at $d = 128$ a dense convolution covering the same three kernel sizes would cost $638{,}976$ weights per step against $22{,}144$ for the separable unit.

The bottleneck. Equation (3) has a consequence: once the convolutions are separable, the *pointwise* term dominates. At $d = 128$ the three depthwise convolutions of one unit cost $5{,}376$ weights while the single 1×1 costs $16{,}512$ — three times all three depthwise convolutions combined. We therefore factorise that 1×1 through a narrow middle layer of width $r = d/4$, squeezing and then expanding:

$$P = P_{\text{expand}}\, P_{\text{squeeze}}, \qquad P_{\text{squeeze}} \in \mathbb{R}^{r \times d}, \quad P_{\text{expand}} \in \mathbb{R}^{d \times r}, \qquad \text{cost } d^2 \to 2dr = d^2/2. \tag{4}$$

The construction is the standard squeeze-and-expand bottleneck (He et al., 2016; Iandola et al., 2016; Sandler et al., 2018); what matters here is where it is applied and what it leaves intact. It halves the dominant term independently of d, and the block still consumes and returns (d, T), so length preservation — and with it the per-step interpretation — is untouched. At the setting used throughout ($d = 64$, 4 blocks) it takes the complete model from 57,580 to 41,324 parameters and from 109.7 to 76.7 MFLOPs per series: $28\,\%$ of the parameters and $30\,\%$ of the arithmetic, for a change that is invisible to the head.

3.3 TOP-k CONJUNCTIVE POOLING

The head is where limitations L1 and L3 are addressed, at zero parameter cost. First we give each class its own gate, $a_j \to a_j^k$, so that classes may disagree about where to look (L1). Then, instead of the fixed mean of Equation (2), we average only the largest evidence values for each class, with the count set as a fraction κ of the series length so that the head is length-agnostic:

$$k = \max\big(1, \lceil \kappa T \rceil\big), \qquad \hat{y}^{\,k} = \frac{1}{k} \sum_{j \in \mathcal{S}^k_{\text{top}}} g_j^k, \qquad \mathbf{I}_{k,j} = g_j^k, \tag{5}$$

where $\mathcal{S}^k_{\text{top}}$ indexes the k largest g_j^k. Selection of the strongest instances is not new in MIL (Durand et al., 2016); the form here is per-class, expressed as a fraction of length, and applied to conjunctive evidence, which gives it a property the sweep in Section 5.4 depends on.

Property 1 (Exact reduction). *At $\kappa = 1$, Equation (5) is class-wise conjunctive pooling term for term. For $\kappa < 1$ the gradient reaches only the selected steps.*

Two consequences. Practically, κ is a hyperparameter and not a weight, so this head has *exactly* the same parameter count as the head it generalises — the interpretation quality it buys is free in

every cost column of Table 2. Experimentally, the $\kappa = 1$ row of Table 3 is not a different model but the same model with selection switched off, so any difference along that column is attributable to selection and to nothing else.

Training objective. Every model is trained with cross-entropy on the bag logits plus one term that exists for the explanation rather than for the classification:

$$\mathcal{L} = \mathrm{CE}(\hat{y}, y) \; + \; \lambda_{\mathrm{focus}}\Big(-\textstyle\sum_j \bar{a}_j \log(\bar{a}_j + \epsilon)\Big), \tag{6}$$

with $\bar{a}_j$ the class-averaged attention. Attention spread evenly over a series has high entropy, so adding entropy to a minimised loss pushes the model towards concentrated attention. Without it a model can be right while its importance map is flat, and a reader cannot tell where the evidence stops. We set $\lambda_{\mathrm{focus}} = 0.01$ for our models and 0 for the MILLET baseline, since the term is our addition; Section 5.4 reports what it is worth.

4 Experimental Setup

Research questions. **RQ1** Does the small model retain classification accuracy? **RQ2** Does it retain a useful interpretation? **RQ3** How much cost is actually removed? **RQ4** What does the bottleneck contribute? **RQ5** What does Top-k pooling contribute?

Datasets. WebTraffic is the synthetic benchmark introduced with MILLET (Early et al., 2024): 1,000 series (500 train / 500 test) of length 1,008 over 10 classes, in which a generator injects the class signal into a known window. It is the only dataset here with *per-time-step ground truth*, so it is the only place where an interpretation can be scored rather than admired, and RQ2 lives there. Breadth comes from the UCR archive (Dau et al., 2019): we report the 85-dataset subset used by MILLET for comparability, and the full 128-dataset archive in Section C.2.

Metrics. Classification is test accuracy, and across UCR also mean rank over the datasets completed by every model in the comparison, with a Wilcoxon–Holm critical-difference analysis (Demšar, 2006) in Section C.2. Interpretation is measured two ways. **AOPCR** (Samek et al., 2017) deletes time steps in the order the model considers important and integrates the drop in the predicted-class score; it needs no ground truth, so it is available everywhere, but it is unnormalised — a point Section 5.5 returns to. **NDCG@n** (Järvelin & Kekäläinen, 2002) ranks the time steps by claimed importance and scores that ranking against the generator's true signal window; it lies in $[0, 1]$ and needs ground truth, so it exists only on WebTraffic. NDCG@n is the stronger evidence of the two and we treat it as such.

Baselines. MILLET with its reference InceptionTime backbone (Ismail Fawaz et al., 2020) is the model this paper is about; FCN and ResNet (Wang et al., 2017) with the same head give two further reference points. Because Section 5.5 shows that faithfulness numbers cannot be moved between training recipes, we retrain every baseline ourselves rather than quoting published values, and we report MILLET twice: once under *our* budget, which is the only fair comparison, and once under *its own* published 1,500-epoch recipe, which is the harder target.

Protocol. One configuration is used for every model and every dataset, so that a difference between two rows is a difference between two models: Adam (Kingma & Ba, 2015) at learning rate 1.25×10^{-3}, weight decay 1.2×10^{-4}, at most 400 epochs with early stopping at patience 60, batch size $\min(\lfloor n_{\mathrm{train}}/10 \rfloor, 16)$ floored at 2, label smoothing 0.13, stratified $80/20$ validation split when $n_{\mathrm{train}} \geq 100$. Encoders use $d = 64$ with 4 blocks unless labelled *wide* ($d = 128$, 6 blocks), and $\kappa = 0.1$; the remaining settings are in Section B.3. Cost is profiled separately at batch 32, length 1008, 10 classes on one GPU: parameters counted directly, FLOPs analytically per series with one multiply–accumulate counted as two operations, latency as the mean of 30 timed passes after 30 discarded warm-up passes.

Two things to hold against every number that follows. First, WebTraffic was also the set on which the design space was screened, so WebTraffic results are optimistic by construction; UCR was never used for selection. Second, seed coverage is uneven: three models were run with three seeds (Table 1, column S) and the rest with one, because the sweep covered 72 encoder–head combinations over up

Table 1: Main comparison. WebTraffic has per-time-step ground truth; UCR-85 is the breadth check and was never used for model selection. S = seeds; multi-seed rows are means. Cost is at batch 32, length 1008, 10 classes. ‡84 of 85 datasets completed. †MILLET's own 1,500-epoch recipe: a harder accuracy target, and an AOPCR value *not* comparable with the rest of that column (Section 5.5). Bold marks the best matched-budget row per column.

		WebTraffic			UCR-85	Cost	
Model	S	Acc. ↑	AOPCR ↑	NDCG@n ↑	Acc. ↑	Params	MFLOPs
FCN + conjunctive	1	0.742	3.826	0.533	0.814	267,035	535.2
ResNet + conjunctive	1	0.772	2.952	0.554	0.815	506,331	1013.2
MILLET (matched budget)	3	0.887	2.569	0.677	**0.827**‡	423,707	847.6
MILLET (published recipe)†	1	0.920	13.268	0.661	0.843‡	423,707	847.6
SEA-Net wide + class-wise	1	0.946	1.722	0.686	**0.829**	269,164	523.7
SEA-Net gated + Top-k	3	**0.955**	2.225	0.750	0.808	61,740	109.7
SEA-Net bottleneck + Top-k (ours)	3	0.947	**2.621**	**0.772**	0.810	**41,324**	**76.7**

Table 2: Cost, profiled at batch 32, length 1008. Weight memory is *computed* from the parameter count (4 B and 1 B per weight); nothing was quantised or deployed. Latency and peak memory are GPU measurements and are not device estimates.

Model	Params	Weights (KB)		MFLOPs	Lat.	Mem.
		fp32	int8		(ms)	(MB)
MILLET	423,707	1655	414	847.6	0.203	112.3
SEA-Net wide	269,164	1051	263	523.7	0.359	123.7
SEA-Net gated	61,740	241	60	109.7	0.128	64.9
SEA-Net bottl.	**41,324**	**161**	**40**	**76.7**	0.131	179.0
SEA-Net bottl., 2 blocks	21,548	84	21	40.0	0.069	178.9

to 129 datasets on a fixed compute allocation. The three-seed standard deviations on our headline model are ± 0.016 for accuracy, ± 0.214 for AOPCR and ± 0.013 for NDCG@n (Section C.1), and we treat any gap smaller than those as uninformative throughout.

5 Results

5.1 RQ1 and RQ2: classification and interpretation at 41 K parameters

Table 1 answers both questions on WebTraffic, in the same direction. Against the baseline trained under our budget, the 41 K model is 6.0 accuracy points better (0.947 against 0.887, three seeds each, seed standard deviation ± 0.016) and 9.5 NDCG@n points better (0.772 against 0.677, ± 0.013); both gaps are several times the seed spread. AOPCR is 2.621 against 2.569, inside the AOPCR standard deviation of either model (± 0.214 and ± 0.884), and should be read as a tie. The honest summary: better classification, better interpretation ranking, unchanged perturbation faithfulness — at $10.3\times$ fewer parameters.

Two rows keep that in proportion. MILLET under its own 1,500-epoch recipe reaches 0.920, so more than half of our margin over the matched-budget baseline is about training budget rather than architecture; our model still leads it by 2.7 accuracy and 11.1 NDCG@n points at $10.3\times$ less cost. And SEA-Net gated + Top-k, at $61{,}740$ parameters, is nominally the most accurate row at 0.955 — but 0.008 against a seed spread of 0.016 is not a difference we claim, and the bottleneck version is preferred for being $33\,\%$ smaller at no cost on NDCG@n.

5.2 RQ3: what the cost reduction actually is

Table 2 separates what improved from what did not. Parameters fall $10.3\times$ and arithmetic $11.1\times$; at 32-bit precision the weights go from about 1.6 MB to 161 KB, which moves the model from above to comfortably below the roughly 1 MB flash budget of the microcontroller class described by Lin et al.

Table 3: Ablations on WebTraffic, all at $d = 64$, 4 blocks, seed 0. *Left:* the two components on and off — reading down gives the bottleneck, across gives Top-k. *Right:* only κ changes, and at $\kappa = 1$ the head *is* the class-wise baseline (Property 1); the last row instead switches off the entropy term of Equation (6).

Encoder	**Head**	Params	Acc.	AOPCR	NDCG@n
plain	class-wise	57,580	0.916	2.478	0.719
plain	Top-k	57,580	0.932	2.691	**0.778**
bottleneck	class-wise	**41,324**	0.902	2.572	0.733
bottleneck	Top-k	**41,324**	**0.938**	**2.778**	0.777

κ	Acc.	AOPCR	NDCG@n
0.05	0.888	2.288	0.715
0.10	0.938	2.778	**0.777**
0.25	**0.940**	2.648	0.733
0.50	0.882	**3.114**	0.732
1.00 *(= baseline head)*	0.908	2.436	0.722
$\lambda_{\text{focus}} = 0$	0.950	2.303	0.765

(2020) and Banbury et al. (2021), and standard int8 quantisation (Jacob et al., 2018) would take it to about 40 KB. That is the sense in which it becomes a candidate for such a device.

The other two columns do not improve, and we state them rather than omit them. Latency improves by only $1.6\times$ despite the $11.1\times$ FLOP reduction, because depthwise convolutions are memory-bound and a GPU at batch 32 is the wrong instrument for a claim about small hardware — the 269 K wide model is in fact the *slowest* row. Peak allocator memory is 179.0 MB against the baseline's 112.3 MB, which is worse, and structurally so: the encoder is length-preserving by design, so activations are (d, T) at every stage regardless of how few weights produce them, and Equation (4) reduces weights, not activations. Since SRAM for activations binds on microcontrollers at least as often as flash for weights, this is a real limitation of the design and not a measurement artefact (Section 6).

5.3 RQ1 ACROSS 85 DATASETS: WHERE SHRINKING STARTS TO COST

UCR was never used for selection, and it is where the trade-off becomes visible. Our 41 K model reaches 0.810 mean accuracy against 0.827 for the matched-budget baseline and 0.843 for MILLET's published recipe: 1.8 and 3.4 points behind, with the same ordering in mean rank over the 84 shared datasets, 15.40 against 12.60 and 9.41 (Section C.2).

What that deficit is made of matters more than its size, and one row of Table 1 answers it. The *wide* version of the same encoder — identical blocks and head family, d raised from 64 to 128 and 4 blocks to 6 — reaches 0.829 and mean rank 11.53, ahead of the matched-budget baseline on both. The architecture is therefore not what loses on UCR; the capacity is. Two effect sizes follow, and they are the practically useful numbers here: changing the backbone at comparable capacity is worth about 1.1 rank places, while moving from our 400-epoch budget to MILLET's 1,500-epoch budget on a *fixed* architecture is worth about 3.2 — three times more. We read this as a priced trade rather than a failure: two accuracy points for an order of magnitude of cost.

5.4 RQ4 AND RQ5: WHICH COMPONENT DID WHAT

RQ4, the bottleneck. Reading down Table 3, it removes 16,256 parameters — 28 % of the model — and accuracy moves by -0.014 with the class-wise head and $+0.006$ with Top-k, both inside the ± 0.016 seed spread; NDCG@n moves by $+0.014$ and -0.001. The defensible statement is therefore that the bottleneck buys the size reduction *at no measurable accuracy cost*, not that it improves anything: a compression the rest of the model does not notice.

RQ5, Top-k pooling. Reading across, NDCG@n rises by 0.059 on the plain encoder and 0.044 on the bottleneck encoder. Against a NDCG@n seed spread of ± 0.013 these are three to four standard deviations, and they are the claim we make for the head; the accuracy movements ($+0.016$ and $+0.036$) point the same way but sit at or below one seed spread, so we do not present Top-k as an accuracy improvement. The right panel of Table 3 isolates the mechanism, and here Property 1 earns its place: the $\kappa = 1$ row is the baseline head exactly, so the 0.777 against 0.722 gap between $\kappa = 0.1$ and $\kappa = 1$ is caused by selection alone. Two features of that table are worth stating plainly rather than smoothing. There is an *optimum*, not a monotone gain — $\kappa = 0.05$ is worse than $\kappa = 0.10$ on every column, so selecting too little is as harmful as selecting nothing and κ has to be set. And AOPCR does not track κ at all, peaking at $\kappa = 0.50$ where NDCG@n is near its minimum. The final row shows the attention-entropy term is a second, independent lever in the same direction: switching

it off improves accuracy (0.950) while costing NDCG@n (0.765) and AOPCR (2.303). Every row is a single seed, and we read the table for shape rather than for individual values.

5.5 A CAVEAT ABOUT AOPCR

One number in Table 1 needs an explanation, and the explanation generalises. MILLET under its published recipe scores AOPCR 13.268 on WebTraffic, five times every other row, on the same architecture, code, metric implementation and data as the matched-budget MILLET row, which scores 2.569; only the training recipe differs. On UCR-85 the same pair gives 0.812 against 3.926, a factor of 4.84. The cause is that AOPCR integrates an *unnormalised* drop in logit (Section A): a model trained longer and to lower loss has sharper logits, so perturbation produces a larger absolute drop whether or not the explanation is better localised. Consistently, the same pair moves by only 1.6 points of NDCG@n — the normalised measure — and moves *downwards*, 0.677 to 0.661, while AOPCR multiplies by 5.2. So an AOPCR value from one paper cannot be compared with one from another: it should be reported only between models trained under a single recipe, or replaced by a normalised measure wherever ground truth allows. That is why we retrain every baseline instead of quoting published numbers, and never compare against MILLET's published AOPCR — including where it would flatter us.

6 DISCUSSION AND LIMITATIONS

What the trade actually is. Section 5 in one sentence: an order of magnitude of parameters and arithmetic, in exchange for about two accuracy points on a broad archive, with the interpretation improved rather than degraded where it can be scored. Whether that is a good trade is a property of the deployment, not of the model. On a workstation it is a bad one. On a sensor node with a megabyte of flash it is the difference between a method that can be considered and one that cannot, and the interpretation, which is the reason to use MIL at all, arrives intact. Since the wide encoder recovers the UCR deficit (Section 5.3), what is being paid for is the size, and the payment can be adjusted by choosing a width. The wider point: the tension between "interpretable" and "small" was never intrinsic to MIL — its head costs $0.25\,\%$ of the baseline — but inherited from a backbone chosen when cost was not a design variable.

Limitations. **No deployment**: we report parameter counts, analytically counted FLOPs and GPU timings; no model was quantised, compiled or run on a microcontroller, and the int8 column of Table 2 is arithmetic rather than measurement. **Activation memory did not shrink**: peak memory is worse than the baseline's (Section 5.2) because the encoder preserves length by design, so where SRAM rather than flash binds this is the wrong design, and streaming or strided variants that keep a per-step interpretation are the obvious next step. **Interpretation is scored on one dataset**: NDCG@n needs per-time-step ground truth, which only the synthetic WebTraffic generator supplies, and that is also the screening set — so the interpretation claim rests on one optimistic benchmark, with AOPCR the weaker fallback (Section 5.5). **Uneven seeds**: three models have three seeds and the ablations are single-seed, read for shape against the spreads in Section 4; a full multi-seed sweep over 72 combinations was outside the available compute. **Training is slower**: 117.3 ± 14.0 s to converge on WebTraffic against 50.4 ± 9.5 s for the baseline, because depthwise convolutions are poorly served by GPU kernels tuned for dense ones. The saving is at inference, where the target deployment spends its life, but the training cost is real.

7 CONCLUSION

Multiple instance learning gives a time series classifier a prediction and a per-time-step explanation from one forward pass, but the reference formulation carries a backbone one to two orders of magnitude larger than the devices that most need both. We showed that the expense is the encoder and not the interpretability, and that a bottlenecked separable dilated encoder with per-class Top-k pooling gives a 41 K-parameter, 76.7 MFLOP model that beats the equally-trained baseline at both tasks where ground truth exists, and trails it by two accuracy points across 85 UCR datasets. Inherent interpretability is not what makes these models large, and does not have to be given up to make them small.

Reproducibility Statement

Everything needed to reproduce the results is either in the paper or in the anonymous code release. The models are defined in Section 3 and, for the encoder and pooling variants not used in the main text, in Sections B.1 and B.2. The single training configuration applied to every model and every dataset is listed in Section 4 and repeated with all remaining hyperparameters in Section B.3. The two evaluation metrics are defined in Section 4, and the perturbation procedure behind AOPCR, which Section 5.5 shows is sensitive to the training recipe, is specified in Section A. WebTraffic is generated by the released generator of Early et al. (2024) at the settings in Section B.3; the UCR archive (Dau et al., 2019) is public and used with its published train/test splits and no additional preprocessing beyond per-series standardisation. Seed counts are reported per row in Table 1 and per-seed values are in Section C.1. Cost figures come from the profiling procedure in Section 4; the script that produces them is in the code release. The full 72-model sweep is reproduced row by row in Section C.5. Anonymous code, configurations and result tables: https://anonymous.4open.science/r/seanet-ANON.

Ethics Statement

This work introduces no new data collection and involves no human subjects. It uses the public UCR archive (Dau et al., 2019) under its original terms and a synthetic benchmark generated by released code; some UCR datasets are derived from human physiological recordings, but they are distributed in anonymised form by their maintainers and we did not re-identify or augment them. The one risk worth naming is specific to the topic: a model that presents an explanation invites more trust than one that does not, and Section 5.5 shows that a common faithfulness score can be inflated by training longer rather than by explaining better. Interpretations from models of this kind should therefore be validated on the deployment domain before being used to support decisions about people. The code we build on is released under the Apache 2.0 licence and its attribution notice is preserved in our release.

AI Use Statement

In this work, we used generative AI tools for writing and editing the text of this paper, for literature search assistance while preparing Section 2, and for producing supporting experiment-management and plotting code. We have not used generative AI tools to generate, select, filter or interpret experimental results, to design the models or the experimental protocol, or to produce any of the numbers reported in this paper. We have reviewed all AI-assisted work. Every numeric value in the paper was traced by the authors to the specific result file and column it came from before it was written down, and the tables were checked against those files a second time after the paper was complete; AI-assisted code was read and tested by the authors, and all reported measurements come from runs the authors executed themselves. Every reference was checked against the original publication, and claims of novelty were checked against a manual literature survey rather than against tool output. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

References

Mohammadreza Baharani and Hamed Tabkhi. ATCN: Resource-efficient processing of time series on edge. *ACM Transactions on Embedded Computing Systems*, 21(5):1–21, 2022.

Colby Banbury, Chuteng Zhou, Igor Fedorov, Ramon Matas, Urmish Thakker, Dibakar Gope, Vijay Janapa Reddi, Matthew Mattina, and Paul Whatmough. MicroNets: Neural network architectures for deploying TinyML applications on commodity microcontrollers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.

François Chollet. Xception: Deep learning with depthwise separable convolutions. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1251–1258, 2017.

Hoang Anh Dau, Anthony Bagnall, Kaveh Kamgar, Chin-Chia Michael Yeh, Yan Zhu, Shaghayegh Gharghabi, Chotirat Ann Ratanamahatana, and Eamonn Keogh. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6):1293–1305, 2019.

Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

Thomas G. Dietterich, Richard H. Lathrop, and Tomás Lozano-Pérez. Solving the multiple instance problem with axis-parallel rectangles. *Artificial Intelligence*, 89(1-2):31–71, 1997.

Thibaut Durand, Nicolas Thome, and Matthieu Cord. WELDON: Weakly supervised learning of deep convolutional neural networks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4743–4752, 2016.

Joseph Early, Gavin K. C. Cheung, Kurt Cutajar, Hanting Xie, Jasmina Kandola, and Niall Twomey. Inherently interpretable time series classification via multiple instance learning. In *International Conference on Learning Representations (ICLR)*, 2024.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size. *arXiv preprint arXiv:1602.07360*, 2016.

Maximilian Ilse, Jakub M. Tomczak, and Max Welling. Attention-based deep multiple instance learning. In *International Conference on Machine Learning (ICML)*, pp. 2127–2136, 2018.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Deep learning for time series classification: A review. *Data Mining and Knowledge Discovery*, 33(4):917–963, 2019.

Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, and François Petitjean. Inceptiontime: Finding AlexNet for time series classification. *Data Mining and Knowledge Discovery*, 34(6):1936–1962, 2020.

Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Bin Li, Yin Li, and Kevin W. Eliceiri. Dual-stream multiple instance learning network for whole slide image classification with self-supervised contrastive learning. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14318–14328, 2021.

Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. MCUNet: Tiny deep learning on IoT devices. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4765–4774, 2017.

Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should I trust you?": Explaining the predictions of any classifier. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1135–1144, 2016.

Matteo Risso, Alessio Burrello, Francesco Conti, Lorenzo Lamberti, Yukai Chen, Luca Benini, Enrico Macii, Massimo Poncino, and Daniele Jahier Pagliari. Lightweight neural architecture search for temporal convolutional networks at the edge. *IEEE Transactions on Computers*, 72(3): 744–758, 2023.

Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019.

Wojciech Samek, Alexander Binder, Grégoire Montavon, Sebastian Lapuschkin, and Klaus-Robert Müller. Evaluating the visualization of what a deep neural network has learned. *IEEE Transactions on Neural Networks and Learning Systems*, 28(11):2660–2673, 2017.

Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4510–4520, 2018.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International Conference on Machine Learning (ICML)*, pp. 3319–3328, 2017.

Zhiguang Wang, Weizhong Yan, and Tim Oates. Time series classification from scratch with deep neural networks: A strong baseline. In *International Joint Conference on Neural Networks (IJCNN)*, pp. 1578–1585, 2017.

Yunshi Wen, Tengfei Ma, Tsui-Wei Weng, Lam M. Nguyen, and Anak Agung Julius. Abstracted shapes as tokens – a generalizable and interpretable model for time-series classification. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://openreview.net/forum?id=pwKkNSuuEs.

Yunshi Wen, Tengfei Ma, Ronny Luss, Debarun Bhattacharjya, Achille Fokoue, and Anak Agung Julius. Shedding light on time series classification using interpretability gated networks. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=n34taxF0TC.

A METRIC DEFINITIONS

AOPCR. Let $\hat{y}^{\,c}(\mathbf{X})$ be the logit of the predicted class c on the unperturbed series, and let $\mathbf{X}^{(m)}$ be the series with the m time steps that the model itself ranks most important for c replaced by the per-series mean. AOPCR integrates the drop over the perturbation order,

$$\mathrm{AOPCR} = \frac{1}{M+1} \sum_{m=0}^{M} \Big(\hat{y}^{\,c}(\mathbf{X}) - \hat{y}^{\,c}\big(\mathbf{X}^{(m)}\big) \Big), \tag{7}$$

following Samek et al. (2017) as adapted to time series by Early et al. (2024), with M set to $10\,\%$ of the series length in steps of $1\,\%$. Higher is better. Note that Equation (7) is a difference of *unnormalised* logits, which is the origin of the comparability problem documented in Section 5.5: two models whose logits live on different scales produce AOPCR values on different scales, whatever the quality of their rankings.

NDCG@n. Where per-time-step ground truth exists, we instead rank the T time steps by the claimed importance $\mathbf{I}_{c,j}$ and score that ranking with normalised discounted cumulative gain (Järvelin & Kekäläinen, 2002) at cut-off n, the number of steps the generator actually marked as carrying the class signal. Relevance is binary. The normalisation is against the ideal ranking of the same series, so NDCG@n lies in $[0, 1]$ and is comparable between models, datasets and papers — which is why the interpretation claims of this paper rest on it and not on AOPCR.

B MODEL AND TRAINING DETAIL

B.1 ENCODER VARIANTS

The main text uses two encoders: the plain multi-scale separable stack of Section 3.2 (`sea_mstcn_sep`) and its bottlenecked form (Equation (4), `sea_mstcn_sep_bottleneck`). Five further variants were swept and appear in Table 4: *gated*, which collapses the time axis to a summary vector and re-weights channels by it; *input-gated*, which reads the same gate from the raw input instead; *spike/trend*, which runs a smoothed and a high-pass branch and mixes them; *reconstruction-residual*, which appends features built from a reconstruction error; and two *wrappers* that do not replace an encoder but sit around one — *multi-scale channels*, which adds rolling max/min/std input channels, and *multi-scale pyramid*, which runs the same encoder at four downsampling rates and fuses back to length T. All preserve T. Only the bottleneck is part of this paper's contribution; the others are the context showing that the bottleneck was the right place to cut.

Table 4: Encoder axis: mean over every head the encoder was paired with, WebTraffic. n is the number of pairings. These are means over heterogeneous pairings, so they indicate spread, not a ranking of encoders.

Encoder	Acc.	AOPCR	NDCG@n	n
multi-scale channels (wrapper)	0.937	2.385	0.747	4
reconstruction-residual	0.924	2.315	0.719	5
input-gated	0.904	2.345	0.718	5
gated	0.902	2.056	0.710	19
plain (`sea_mstcn_sep`)	0.898	1.933	0.694	12
bottleneck	0.884	2.209	0.694	8
spike/trend	0.858	1.814	0.677	7
multi-scale pyramid (wrapper)	0.771	1.394	0.604	3

B.2 POOLING HEADS

Every head keeps the two-branch template of Equation (1), so a head is fully described by two choices: the shape of the attention branch, and the operator that reduces the T evidence values g_j^k to one number. Table 5 lays out all seven on those two axes together with the parameter setting at which each becomes conjunctive pooling exactly. Two are adapted from histopathology MIL rather than designed here: gated attention is due to Ilse et al. (2018) and the dual-stream mechanism to Li et al. (2021); our changes to those two are the transfer to time series, the per-class formulation, and — for the dual stream — removing its contrastive pre-training and adding a learnable blend.

Table 5: The seven heads as two independent slots. "Reduces at" names the setting at which the head becomes conjunctive pooling exactly. L1–L3 are the limitations listed in Section 3.1.

Head	**Attention**	**Reduction over time**	**Reduces at**	**Targets**
Conjunctive (baseline)	shared gate	mean	—	—
Class-wise conjunctive	per-class gate	mean	$a_j^k = a_j$	L1
Softmax conjunctive	per-class score	softmax, learnable τ	$\tau \to \infty$	L2
Adaptive class-wise	per-class gate	soft-argmax, learnable β_k	$\beta_k \to 0$	L3
Top-k conjunctive	per-class gate	mean of the k largest	$\kappa = 1$	L3
Attention-max	per-class gate	blend of mean and max	$\lambda_k \to 0$	L3
Per-class gated attention	gated $\tanh\odot\sigma$	softmax	**none**	L1, L3
Dual-stream conjunctive	gate + critical query	blend of two streams	$\lambda \to 0$	L3

Table 6 averages each head over every encoder it was paired with. Two observations support statements made in the main text. First, the spread of the head choice (0.744 to 0.921) is as large as the spread of the encoder choice in Table 4 (0.771 to 0.937), so the pooling head is a first-class design axis and not a detail. Second, a reduction property is *not* a safety guarantee: five of the seven heads provably contain conjunctive pooling as a special case, and the dual-stream head, which is one of them, is nevertheless the worst head tested — its four pairings score 0.860, 0.824, 0.736 and 0.556. The mechanism is visible in the training runs: its blend weight starts at $\lambda_0 = 0.05$ and nothing in Equation (6) pushes it back towards the reduction point, so being *able* to recover the baseline

does not mean being optimised towards it. That is why the main text argues for Top-k on measured NDCG@n rather than on its reduction property alone.

Table 6: Head axis: mean over every encoder the head was paired with, WebTraffic. Excludes the κ and λ_{focus} ablation configurations and the published-recipe baseline, which would otherwise weight Top-k and the bottleneck encoder.

Head	Acc.	AOPCR	NDCG@n	n
Class-wise conjunctive	0.921	2.322	0.692	10
Per-class gated attention	0.918	1.869	0.733	4
Adaptive class-wise	0.912	2.210	0.732	9
Softmax conjunctive	0.907	1.690	0.712	9
Top-k conjunctive	0.904	2.267	0.712	16
Attention-max	0.838	1.758	0.645	8
Dual-stream conjunctive	0.744	2.007	0.636	4

B.3 Hyperparameters

Identical for every model and every dataset unless stated: Adam (Kingma & Ba, 2015), learning rate 1.25×10^{-3}, weight decay 1.2×10^{-4}, at most 400 epochs, early stopping patience 60 on validation loss, batch size $\min(\lfloor n_{\text{train}}/10 \rfloor, 16)$ with a floor of 2, label smoothing 0.13, stratified $80/20$ validation split when $n_{\text{train}} \geq 100$ and training-set monitoring otherwise. Encoder: $d = 64$, 4 blocks (wide: $d = 128$, 6 blocks), kernels $\{5, 11, 23\}$, dilation doubling per block capped at 16, dropout 0.2. Head: attention width 8, $\kappa = 0.1$, $\tau_0 = 1.0$, $\beta_0 = 0.3$, $\lambda_0 = 0.5$ (attention-max) or 0.05 (dual-stream). Loss: $\lambda_{\text{focus}} = 0.01$ for our models and 0 for the MILLET baselines, since the term is our addition. The published-recipe baseline uses MILLET's own settings instead, of which the material difference is 1,500 epochs against our 400. WebTraffic is generated at the released defaults: 1,000 series, length 1,008, 10 classes, $50/50$ train/test split. Series are standardised per series; no other preprocessing is applied.

C Additional results

C.1 Per-seed values

Table 7: The three models run with three seeds on WebTraffic. The standard deviations in the last three columns are the noise floor used throughout the paper: no difference smaller than the relevant one is reported as a result.

Model	seed 0	seed 1	seed 2	Acc. mean $\pm$ sd	AOPCR sd	NDCG@n sd
SEA-Net gated + Top-k	0.954	0.958	0.952	0.955 ± 0.003	± 0.131	± 0.022
SEA-Net bottleneck + Top-k	0.938	0.938	0.966	0.947 ± 0.016	± 0.214	± 0.013
MILLET (matched budget)	0.894	0.876	0.892	0.887 ± 0.010	± 0.884	± 0.016

The single-seed rows of Table 3 are all instances of the bottleneck Top-k model with one component or one hyperparameter changed, so its spreads are the right noise floor for them. Note that seed 0 of that model gives 0.938, not its three-seed mean of 0.947; Table 3 therefore uses 0.938, so that every cell in both its panels comes from the same seed.

C.2 UCR archive detail

Table 8 is the evidence behind Section 5.3. Two effect sizes are worth isolating. Holding the training budget fixed and changing the backbone at comparable capacity moves the mean rank from 12.60 to 11.53, about 1.1 places. Holding the architecture fixed and changing only the training budget moves it from 12.60 to 9.41, about 3.2 places — three times as much. Any comparison of backbones in this literature that does not hold the budget fixed is measuring mostly the budget.

Figure 3 is the corresponding Wilcoxon–Holm critical-difference analysis (Demšar, 2006). Its mean ranks are systematically about 0.5–0.7 higher than those in Table 8, because it includes MILLET's

Table 8: UCR-85 accuracy and mean rank. Mean rank is over the 84 datasets completed by all 28 fully-swept models; lower is better. Win/tie/loss is per dataset against MILLET's *published* accuracies. ‡84 of 85 datasets completed.

Model	Params	Acc.	Mean rank	W/T/L
MILLET, published (Early et al., 2024)	423,707	0.8445	—	—
MILLET, published recipe, our code	423,707	0.8434‡	**9.41**	26/32/26
SEA-Net wide + class-wise conjunctive	269,164	0.8292	11.53	26/19/39
SEA-Net wide + MILLET additive	269,083	0.8298	11.98	23/19/42
SEA-Net wide + MILLET conjunctive	269,083	0.8238	12.43	22/20/42
MILLET, matched budget	423,707	0.8274‡	12.60	13/25/46
ResNet + conjunctive	506,331	0.8146	13.54	25/11/49
FCN + conjunctive	267,035	0.8141	14.48	20/12/52
SEA-Net gated + Top-k	61,740	0.8083	15.29	19/13/53
SEA-Net bottleneck + Top-k	**41,324**	0.8097	15.40	20/15/50

published per-dataset accuracies as a 29th entry and that shifts every rank; the ordering is unchanged. The diagram also shows that pairwise differences through the middle of the field are not significant at $\alpha = 0.05$ after Holm correction, which is a further reason the main text argues from WebTraffic and from cost rather than from UCR rank.

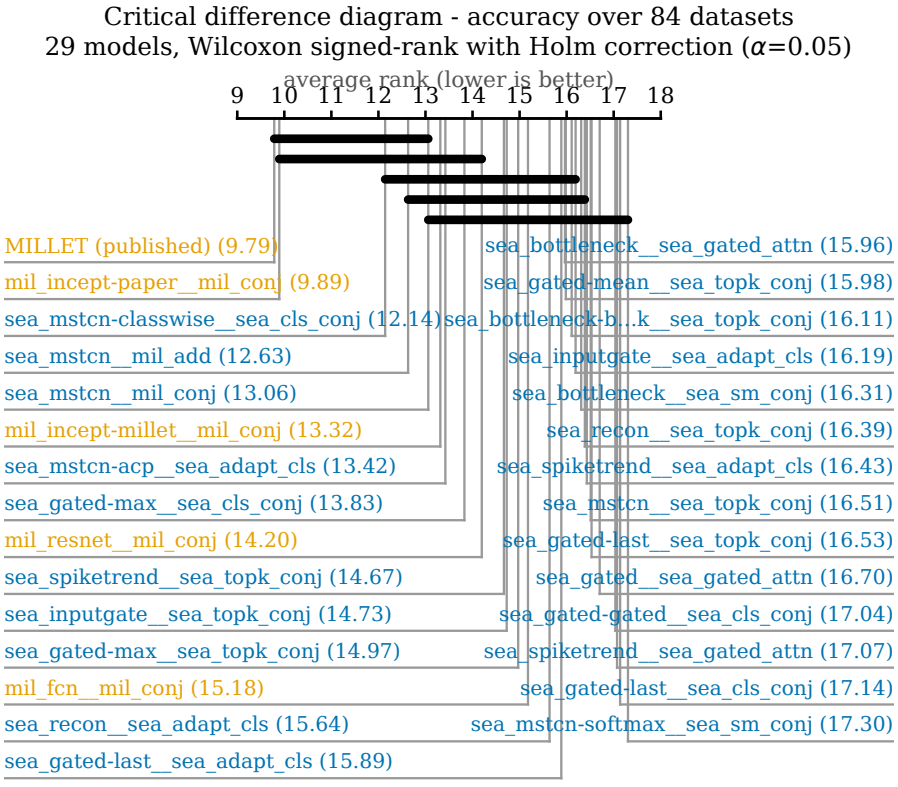

Figure 3: Critical-difference diagram over the shared UCR datasets. Lower rank is better; horizontal bars join models that are not significantly different after Holm correction. Orange labels are reproduced baselines, blue are ours. Ranks differ from Table 8 because this diagram includes one additional entry — see the text above.

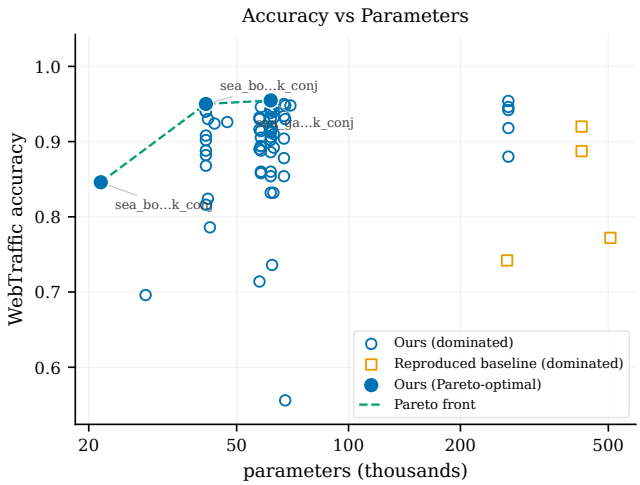

Figure 4: All 72 swept encoder–head combinations (circles) and the reproduced baselines (squares) on WebTraffic. Filled circles are Pareto-optimal: nothing beats them on both axes at once. The baselines occupy the right-hand quarter of the parameter axis without occupying the top of the accuracy axis, which is the picture behind Tables 1 and 2. Labels are abbreviated configuration names; Table 10 gives them in full.

Table 9: Five ways of obtaining a MILLET number, and what each is evidence for.

How obtained	WebTraffic Acc.	UCR-85 Acc.
Published, mean of 5 seeds (Early et al., 2024)	0.924	0.8445
Published, 5-model ensemble	0.940	—
Released weights, evaluated in our pipeline	0.922	—
Re-trained here under *their* hyperparameters	0.920	0.8434
Re-trained here under *our* configuration	0.887	0.8274

C.3 Cost against quality across the whole sweep

C.4 Relation to published MILLET results

The third and fourth rows agree with the first to within 0.4 and 0.1 accuracy points, which is the check that our re-implementation is faithful. The gap between the fourth and fifth rows is therefore the training budget and nothing else, and it is why the main comparison in Table 1 treats the matched-budget row as the like-for-like baseline while still printing the published-recipe row as the harder target. We do not reproduce MILLET's published AOPCR in this table: Section 5.5 shows it would not be comparable with any value we measured, and quoting it would contradict our own finding.

C.5 Full sweep

Table 10 lists all 72 encoder–head combinations, ordered by WebTraffic accuracy. UCR columns are blank where the combination was not run on the archive; those combinations were screened on WebTraffic only, under a fixed compute allocation. One row, `seanet_bottleneck_adaptive`, completed only 6 of the 85 UCR datasets, so its UCR values are omitted rather than reported — a 6-dataset mean is not comparable with an 85-dataset mean.

Table 10: All 72 encoder–head combinations, ordered by WebTraffic accuracy. A dash in the UCR column means the combination was screened on WebTraffic only. Pooling heads marked * are MILLET's own, reused unchanged.

#	**Configuration**	**Encoder**	**Head**	Acc.	AOPCR	NDCG@n	UCR-85	Params
1	`seanet_gated_mean_topk`	gated	Top-k	0.955	2.23	0.750	0.8083	61,740
2	`seanet_conjunctive`	plain	conjunctive*	0.954	1.50	0.698	0.8238	269,083

continued on the next page

Table 10 continued from the previous page.

#	Configuration	Encoder	Head	Acc.	AOPCR	NDCG@n	UCR-85	Params
3	seanet_gated_max_topk	gated	Top-k	0.952	2.27	0.719	0.8108	61,740
4	seanet_topk_nofocus	bottleneck	Top-k	0.950	2.30	0.765	—	41,324
5	seanet_spiketrend_topk	spike/trend	Top-k	0.950	2.32	0.756	0.8089	67,020
6	seanet_inputgate_mschan	ms-channels	Top-k	0.948	2.32	0.757	—	69,768
7	seanet_gated_mschan	ms-channels	Top-k	0.948	2.22	0.717	—	67,592
8	seanet_bottleneck_topk	bottleneck	Top-k	0.947	2.62	0.772	0.8097	41,324
9	seanet_recon_adaptive	recon	adaptive	0.946	2.60	0.768	0.8140	62,935
10	seanet_inputgate_topk	input-gated	Top-k	0.946	2.80	0.748	0.8131	58,092
11	seanet_classwise	plain	class-wise	0.946	1.72	0.686	0.8292	269,164
12	seanet_gated_max	gated	class-wise	0.944	1.89	0.592	0.8237	61,740
13	seanet_gated_last_topk	gated	Top-k	0.942	2.31	0.748	0.8015	61,740
14	seanet	plain	additive*	0.942	1.58	0.729	0.8298	269,083
15	seanet_gated_last	gated	class-wise	0.942	2.52	0.742	0.7888	61,740
16	seanet_recon_topk	recon	Top-k	0.940	2.51	0.698	0.8063	62,925
17	seanet_topk_k025	bottleneck	Top-k	0.940	2.65	0.733	—	41,324
18	seanet_bottleneck_softmax	bottleneck	softmax	0.940	1.68	0.740	0.8106	41,325
19	seanet_gated_last_adaptive	gated	adaptive	0.938	2.39	0.793	0.8029	61,750
20	seanet_spiketrend_adaptive	spike/trend	adaptive	0.934	1.91	0.747	0.8010	67,030
21	seanet_recon_softmax	recon	softmax	0.932	1.64	0.723	—	62,926
22	seanet_slim_topk	plain	Top-k	0.932	2.69	0.778	0.8093	57,580
23	seanet_bottleneck_gatedattn	bottleneck	gated-attn	0.930	2.23	0.741	0.8146	41,844
24	seanet_spiketrend_gatedattn	spike/trend	gated-attn	0.930	1.59	0.721	0.8039	67,540
25	seanet_gated	gated	class-wise	0.930	2.41	0.637	0.7900	61,740
26	seanet_slim_adaptive	plain	adaptive	0.930	2.45	0.793	—	57,590
27	seanet_bottleneck_mschan	ms-channels	Top-k	0.926	2.24	0.777	—	47,176
28	seanet_bottleneck_mschan_lean	ms-channels	Top-k	0.924	2.76	0.735	—	43,576
29	seanet_gated_last_softmax	gated	softmax	0.920	1.99	0.738	—	61,741
30	millet_paper	InceptionTime	conjunctive*	0.920	13.27	0.661	0.8434	423,707
31	seanet_acp	plain	adaptive	0.918	1.61	0.619	0.8134	269,174
32	seanet_gated_last_gatedattn	gated	gated-attn	0.918	1.35	0.757	0.8048	62,260
33	seanet_slim	plain	class-wise	0.916	2.48	0.719	—	57,580
34	seanet_gated_max_softmax	gated	softmax	0.914	1.88	0.776	—	61,741
35	seanet_inputgate	input-gated	class-wise	0.914	2.48	0.715	—	58,092
36	seanet_gated_mean_softmax	gated	softmax	0.912	2.09	0.765	—	61,741
37	seanet_recon	recon	class-wise	0.910	2.50	0.680	—	62,925
38	seanet_topk_k100	bottleneck	Top-k	0.908	2.44	0.722	—	41,324
39	seanet_gated_max_adaptive	gated	adaptive	0.906	2.07	0.729	—	61,750
40	seanet_inputgate_adaptive	input-gated	adaptive	0.905	2.65	0.732	0.8153	58,102
41	seanet_spiketrend	spike/trend	class-wise	0.904	2.15	0.699	—	67,020
42	seanet_bottleneck	bottleneck	class-wise	0.902	2.57	0.733	—	41,324
43	seanet_gated_mean	gated	class-wise	0.902	2.49	0.714	—	61,740
44	seanet_inputgate_softmax	input-gated	softmax	0.894	1.64	0.696	—	58,093
45	seanet_slim_gatedattn	plain	gated-attn	0.894	2.30	0.714	—	58,100
46	seanet_recon_attnmax	recon	attn-max	0.892	2.32	0.725	—	62,935
47	seanet_slim_softmax	plain	softmax	0.890	1.83	0.705	—	57,581
48	seanet_slim_gatedattn	gated	attention*	0.888	2.40	0.710	—	58,100
49	seanet_topk_k005	bottleneck	Top-k	0.888	2.29	0.715	—	41,324
50	millet	InceptionTime	conjunctive*	0.887	2.57	0.677	0.8274	423,707
51	seanet_gated_max_attnmax	gated	attn-max	0.886	1.12	0.575	—	61,750
52	seanet_topk_k050	bottleneck	Top-k	0.882	3.11	0.732	—	41,324
53	seanet_softmax	plain	softmax	0.880	1.02	0.605	0.8131	269,165
54	seanet_spiketrend_softmax	spike/trend	softmax	0.878	1.43	0.665	—	67,021
55	seanet_bottleneck_adaptive	bottleneck	adaptive	0.868	1.97	0.680	—	41,334
56	seanet_slim_dualstream	plain	dual-stream	0.860	2.40	0.745	—	58,101
57	seanet_gated_mean_adaptive	gated	adaptive	0.860	2.24	0.731	—	61,750
58	seanet_inputgate_attnmax	input-gated	attn-max	0.858	2.15	0.700	—	58,102
59	seanet_spiketrend_attnmax	spike/trend	attn-max	0.854	1.65	0.644	—	67,030
60	seanet_gated_last_attnmax	gated	attn-max	0.854	1.78	0.668	—	61,750
61	seanet_bottleneck_shallow	bottleneck	Top-k	0.846	2.82	0.624	—	21,548
62	seanet_gated_mean_attnmax	gated	attn-max	0.832	1.67	0.704	—	61,750
63	seanet_gated_pyramid	ms-pyramid	Top-k	0.832	0.98	0.591	—	62,781
64	seanet_bottleneck_dualstream	bottleneck	dual-stream	0.824	2.04	0.652	—	41,845
65	seanet_bottleneck_attnmax	bottleneck	attn-max	0.816	1.74	0.610	—	41,334
66	seanet_bottleneck_pyramid	ms-pyramid	Top-k	0.786	1.03	0.569	—	42,365
67	resnet	ResNet	conjunctive*	0.772	2.95	0.554	0.8146	506,331
68	fcn	FCN	conjunctive*	0.742	3.83	0.533	0.8141	267,035
69	seanet_gated_last_dualstream	gated	dual-stream	0.736	1.95	0.637	—	62,261
70	seanet_slim_attnmax	plain	attn-max	0.714	1.63	0.532	—	57,590
71	seanet_bottleneck_mschan_pyramid	ms-pyramid	Top-k	0.696	2.17	0.653	—	28,441
72	seanet_spiketrend_dualstream	spike/trend	dual-stream	0.556	1.64	0.510	—	67,541